

Teste de Software – Bad Smells

Análise de Smells

- Complexidade cognitiva alta: muitos ramos (if/else) aninhados no mesmo método. Isso dificulta entender o fluxo e aumenta o risco de regressões ao mudar regras.
- Condicionais aninhadas colapsáveis: havia if dentro de if que podiam ser combinados. Isso espalha a lógica e torna a leitura mais lenta, além de facilitar a criação de casos não cobertos por testes.
- Método longo com múltiplas responsabilidades: o método fazia cabeçalho, corpo e rodapé. Essa mistura reduz coesão, dificulta reutilização e torna os testes mais verbosos porque tudo fica acoplado a um único ponto.

Porque isso é ruim para manutenção e testes?

- Quanto maior a complexidade e o número de caminhos, mais difícil é cobrir tudo com testes e mais provável é esquecer cenários.
- Duplicação de lógica (CSV/HTML em vários pontos) aumenta o custo de mudanças e torna erros “quase iguais” mais difíceis de identificar.
- Baixa coesão impede testes unitários pequenos e claros, porque o comportamento fica concentrado em um único método grande.

Relatório da Ferramenta (ESLint + sonarjs)

O eslint-plugin-sonarjs apontou problemas que poderiam passar numa revisão manual rápida, como **complexidade cognitiva** acima do limite e **condicionais colapsáveis**. Isso ajuda a detectar pontos onde o código está “funcionando”, mas já está difícil de manter.

Processo de Refatoração (smell mais crítico: complexidade cognitiva)

Antes

```
vinic@VXPC MINGW64 ~/OneDrive/Desktop/Teste de Software/bad-smells-js-refactoring (main)
$ npx eslint src/ReportGenerator.js

C:\Users\vinic\OneDrive\Desktop\Teste de Software\bad-smells-js-refactoring\src\ReportGenerator.js
 11:3  error  Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed  sonarjs/cognitive-complexity
 43:14 error  Merge this if statement with the nested one  sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

```

for (const item of items) {
  if (user.role === 'ADMIN') {
    // Admins veem todos os itens
    if (item.value > 1000) {
      // Lógica bônus para admins
      item.priority = true;
    }

    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? ' style="font-weight:bold;"' : '';
      report += `<tr${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    // Users comuns só veem itens de valor baixo
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
        total += item.value;
      }
    }
  }
}
}

```

Depois

```

for (const item of items) {
  this.applyAdminPriority(user, item);

  if (!this.shouldIncludeItem(user, item)) {
    continue;
  }

  report += this.renderItemRow(reportType, item, user);
  total += item.value;
}

```

Técnicas aplicadas

- Extract Method para separar responsabilidades (buildHeader, buildFooter, shouldIncludeItem, renderItemRow).
- Guard Clause para reduzir aninhamento (if (!shouldIncludeItem) continue).
- Separação de responsabilidades para manter lógica de formatação e regras de negócio mais legíveis e testáveis.

Conclusão

Testes funcionam como rede de segurança, permitem refatorar com confiança, sabendo que o comportamento esperado continua válido. Ao reduzir bad smells, o código fica mais legível, mais fácil de testar e com menor risco de regressão em futuras mudanças.

Resultados obtidos nos comandos “npm test” e “npx eslint src/ReportGeneratorRefactored.js ” após a refatoração:

```
vinic@VXPC MINGW64 ~/OneDrive/Desktop/Teste de Software/bad-smells-js-refactoring (main)
$ npm test

> bad-smells-js-refactoring@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js

(node:38528) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS tests/ReportGenerator.test.js
  ReportGenerator (Rede de Segurança)
    ✓ deve lidar com array de itens vazio corretamente
    Admin User
      ✓ deve gerar um relatório CSV completo para Admin (2 ms)
      ✓ deve gerar um relatório HTML completo para Admin (com prioridade) (1 ms)
    Standard User
      ✓ deve gerar um relatório CSV filtrado para User (apenas itens <= 500) (1 ms)
      ✓ deve gerar um relatório HTML filtrado para User (apenas itens <= 500)
  ReportGeneratorRefactored (Rede de Segurança)
    ✓ deve lidar com array de itens vazio corretamente
    Admin User
      ✓ deve gerar um relatório CSV completo para Admin (1 ms)
      ✓ deve gerar um relatório HTML completo para Admin (com prioridade)
    Standard User
      ✓ deve gerar um relatório CSV filtrado para User (apenas itens <= 500)
      ✓ deve gerar um relatório HTML filtrado para User (apenas itens <= 500)

Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        0.219 s, estimated 1 s
Ran all test suites.
```

```
vinic@VXPC MINGW64 ~/OneDrive/Desktop/Teste de Software/bad-smells-js-refactoring (main)
$ npx eslint src/ReportGeneratorRefactored.js
```